HOW I LEARNED TO LOVE THE SITUATION CALCULUS

Raymond Reiter
Department of Computer Science
University of Toronto
and
The Canadian Institute for Advanced Research

The Situation Calculus (McCarthy)

- A first order language for representing dynamically changing worlds; all changes are the result of named $actions$.
- The world is conceived as being in some state s; this state can change only in consequence of some agent performing an action.
- $do(\alpha, s)$ denotes the successor state to s resulting from performing the action α.
- Actions may be parameterized:
  $put(x, y)$ might stand for the action of putting object x on object y; $do(put(A, B), s)$ denotes that state resulting from placing A on B when the world is in state s.
- Actions are denoted by function symbols.
- $Fluents =$ those relations whose truth values may vary from state to state.
  - Denoted by predicate symbols taking a state term as one of their arguments.
  - In a world in which it is possible to paint objects, we might have a fluent $colour(x, c, s)$, meaning that the colour of object x is c when the world is in state s.

The Situation Calculus (Continued)

- Actions have *preconditions*: sufficient conditions which a world state must satisfy before the action can be performed in this state.
  - It is possible for a robot r to pick up an object x in the world state s provided the robot is not holding any object, it is next to x, and x is not heavy:

$$[(\forall z)\neg holding(r, z, s)] \wedge \neg heavy(x) \wedge nexto(r, x, s) \supset Poss(pickup(r, x), s).$$

  - It is possible for a robot to repair an object provided the object is broken, and there is glue available:

$$hasglue(r, s) \wedge broken(x, s) \supset Poss(repair(r, x), s).$$

- World dynamics are specified by *effect axioms* which specify the effect of a given action on the truth value of a given fluent.
  - The effect on the fluent $broken$ of a robot dropping an object:

$$Poss(drop(r, x), s) \wedge fragile(x) \supset broken(x, do(drop(r, x), s)).$$

  - A robot repairing an object causes it not to be broken:

$$Poss(repair(r, x), s) \supset \neg broken(x, do(repair(r, x), s)).$$

The Frame Problem (McCarthy and Hayes)

- Axioms other than effect axioms are required for formalizing dynamic worlds. These are called $frame\ axioms$, and they specify the action $invariants$ of the domain, i.e., those fluents unaffected by the performance of an action.
  - Dropping things does not affect an object's colour:

$$Poss(drop(r, x), s) \wedge colour(y, c, s) \supset colour(y, c, do(drop(r, x), s)).$$

  - Not breaking things:

$$Poss(drop(r, x), s) \wedge \neg broken(y, s) \wedge [y \neq x \vee \neg fragile(y)] \supset \neg broken(y, do(drop(r, x), s)).$$

- Problem: Vast number of frame axioms; only relatively few actions will affect the truth value of a given fluent. All other actions leave the fluent invariant.
  - An object's colour remains unchanged as a result of picking things up, opening a door, turning on a light, electing a new prime minister of Canada, etc. etc.
- $\sim 2 \times \mathcal{A} \times \mathcal{F}$ frame axioms.

What Counts as a Solution to the FP?

- Suppose the person responsible for axiomatizing an application domain has suceeded in writing down **all** the effect axioms, i.e. for each fluent F and each action A which can cause F's truth value to change, an axiom of the form

$$Poss(A, s) \wedge R(s) \supset \pm F(do(A, s)).$$

- Want a systematic procedure for generating, from these effect axioms, all the frame axioms.
- If possible, also want a $parsimonious$ representation for these frame axioms (because in their simplest form, there are too many of them).

Why Do We Want a Solution to the FP?

- Frame axioms are necessary to reason about the domain being formalized.
- Convenience of the axiomatizer.
  - Modularity. The axiomatizer needs only to add new effect axioms.
  - Accuracy. No inadvertent omissions of frame axioms.

The Situation Calculus in AI

- Historically, the situation calculus was invented as a $planning$ formalism; until very recently, this has been its only role.
- The idea is to represent a dynamic world as situation calculus axioms. To obtain a plan whose execution would lead to a world in which the goal $G(s)$ would be true, show that

$$Axioms \models (\exists\, s)G(s).$$

- Any binding for s resulting from such a proof is a plan for this task (c.f. Prolog).

$do(put(A, B), do(pickup(A, do(put(B, C), do(pickup(B), S_0)))))$.

The Current Status of the Situation Calculus

- In response to the "Yale shooting problem", a large body of research has been done in the past six years on the frame and related problems.
- Appeals to various nonmonotonic logics (Circumscription, Default Logic, Autoepistemic Logic).
- New problems uncovered, e.g. the *ramification problem*.
- New techniques invented, old techniques refined:
  - Proofs of properties of states by mathematical induction.
  - Soundness and completeness results for goal regression as a plan synthesis technique.
  - Proofs of correctness of various solutions to the frame problem.
- The ontology of the situation calculus extended to include:
  - Time.
  - Concurrent actions.
  - Actions with durations.
  - External events.
  - Complex actions.
- Many problems remain, but the theory is now more or less sufficiently well understood to provide an off-the-shelf formalism.

Other Uses for the Situation Calculus

- Formalizing update transactions in database theory.
- Software specification (with Alex Borgida and John Mylopoulos).
- Complex events for discrete event simulation and "knowledge-based" control systems (with Hector Levesque and Fangzhen Lin).
- In general, any branch of Computer Science concerned with $dynamics$ – and what isn't? – can profit from the theoretical foundations for the situation calculus provided by AI researchers.

Formalizing Database Update Transactions in the Situation Calculus

Formalizing Databases in the Situation Calculus

Motivation and Background

- Databases evolve over time as a result of *transactions*, whose purpose is to update the database with new information.
- Example: An educational database might have a transaction specifically designed to change a student's grade.
  - This would normally be a procedure which, when invoked on a specific student and grade, first checks that the database satisfies certain preconditions (e.g., that there is a record for the student, and that the new grade differs from the old), and if so, records the new grade.
- This is a *procedural* notion. Transactions also *physically modify* the database.
- Ideally, we want a *specification* of the entire evolution of a database.
- Proposal: Specify database transactions by appealing to various ideas from the AI planning literature.
  - The situation calculus.
  - The frame problem.
  - The projection problem in planning.
  - Goal regression for plan synthesis.
- Theory of database updates $\cong$ AI planning in the situation calculus.

Database Updates: A Proposal

- Represent databases in the situation calculus. Updatable relations are fluents, i.e. they take a state argument.
- Treat update transactions exactly like actions in the AI planning domain. Transactions are functions.
- For this to work, need a solution to the frame problem.

The Basic Approach: An Example

An Education Database

- **Relations**
  1. $enrolled(st, course, s)$: st is enrolled in $course$ when the database is in state s.
  2. $grade(st, course, grade, s)$: The grade of st in $course$ is $grade$ when the database is in state s.
  3. $prerequ(pre, course)$: pre is a prerequisite course for $course$.
- **Initial Database State**
  These can be arbitrary first order sentences, the only restriction being that fluents mention only the initial state S_0.

$$enrolled(Sue, C100, S_0) \vee enrolled(Sue, C200, S_0),$$

$$(\exists\, c) enrolled(Bill, c, S_0),$$

$$(\forall\, p).prerequ(p, P300) \equiv p = P100 \vee p = M100,$$

$$(\forall\, p) \neg prerequ(p, C100),$$

$$(\forall\, c).enrolled(Bill, c, S_0) \equiv c = M100 \vee c = C100 \vee c = P200,$$

$$enrolled(Mary, C100, S_0), \quad \neg enrolled(John, M200, S_0), \ldots$$

$$grade(Sue, P300, 75, S_0), \quad grade(Bill, M200, 70, S_0), \ldots$$

Example: Continued

- **Database Transactions**
  - Denote by function symbols.
  - Treat exactly like actions in situation calculus planning.
- For the example, there are three transactions:
  1. $register(st, c)$,
  2. $change(st, c, g)$,
  3. $drop(st, c)$.
- A student can register in a course iff she has obtained a grade of at least 50 in all prerequisites for the course:

$$Poss(register(st, c), s) \equiv \{(\forall\, p).prerequ(p, c) \supset (\exists\, g).grade(st, p, g, s) \wedge g \geq 50\}.$$

- It is possible to change a student's grade iff he has a grade which is different than the new grade:

$$Poss(change(st, c, g), s) \equiv (\exists\, g').grade(st, c, g', s) \wedge g' \neq g.$$

- A student may drop a course iff the student is currently enrolled in that course:

$$Poss(drop(st, c), s) \equiv enrolled(st, c, s).$$

Example: Continued

- **Update Specifications**

$$Poss(a, s) \supset [enrolled(st, c, do(a, s)) \equiv a = register(st, c) \vee enrolled(st, c, s) \wedge a \neq drop(st, c)],$$

$$Poss(a, s) \supset [grade(st, c, g, do(a, s)) \equiv a = change(st, c, g) \vee grade(st, c, g, s) \wedge (\forall g')a \neq change(st, c, g')].$$

- The update specification axioms "solve" the frame problem. Why?
  - Let α be *any* transaction distinct from $register(st, c)$ and $drop(st, c)$.
  - We obtain the following instance of the first axiom:

    $$Poss(\alpha, s) \supset [enrolled(st, c, do(\alpha, s)) \equiv enrolled(st, c, s)],$$

    i.e., $register(st, c)$ and $drop(st, c)$ are the only transactions which can possibly affect the truth value of $enrolled$; *all other transactions leave its truth value unchanged.*
  - Requires unique names axioms for transactions:

    $$change(st, c, g) \neq drop(st, c),$$

    $$drop(st, c) \neq register(st, c),$$

    $$etc.$$

Queries

- All updates are $virtual$; the database is never physically changed.
- Querying the database resulting from a sequence of update transactions:
  "Is John enrolled in any courses after the transaction sequence $drop(John, C100), register(Mary, C100)$ has been 'executed'?"

  $$Database \models (\exists\, c).enrolled(John, c, do(register(Mary, C100), do(drop(John, C100), S_0))).$$

- This is the $projection\ problem$ in AI planning.
- A sound and complete solution to the projection problem $=$ a sound and complete query evaluation mechanism for evolving databases.

Some Results on Evolving Databases

- Goal regression (a classical AI planning technique) provides a sound and complete query evaluator.
- A Prolog implementation of such a query evaluator.
- Integrity constraints as inductive entailments of the situation calculus axiomatization.

R. Reiter. On Specifying Database Updates. Technical report, Department of Computer Science, University of Toronto, 1992.

R. Reiter. The Frame Problem in the Situation Calculus: A Simple Solution (Sometimes) and a Completeness Result for Goal Regression. *Artificial Intelligence and Mathematical Theory of Computation: Papers in Honor of John Mccarthy*, Vladimir Lifschitz (ed.), Academic Press, San Diego, CA, 1991, pp.359-380.

Software Specification

Joint work with with Alex Borgida and John Mylopoulos.

Software Specification

- The idea here is to provide task *specifications* in some suitable formal language, without any commitment to how these tasks are to be eventually implemented.
- **Example**: A specification in System Z
  In a university environment, suppose we want to specify an operation to enrol a student in a course.
  - If there is room in the course, the student is added to the roster, and the count of spaces left is decremented.
  - If this count is zero, and the student is in her last year of school, her request is added to a waiting list.
  - Otherwise, the student is sent a notice rejecting her application.

  Abstractly, this could be represented by a simplified Z specification where we start with the state description

$$
\begin{array}{|l}
\hline
CourseRequests \\
\hline
r : ROSTER \\
c : COUNTER \\
w : WAITING \\
n : NEGATIVE - LETTER \\
\hline
\end{array}
$$

Software Specification (Continued)

- Specification of the procedure $Enrol$

$$
\begin{array}{|l}
\hline
Enrol \\
\Delta CourseRequests \\
\hline
\neg\, full \Rightarrow r' = r + 1 \land c' = c - 1 \\
(full \land lastYear) \Rightarrow w' = w + 1 \\
(full \land \neg\, lastYear) \Rightarrow n' = n + 1 \\
\hline
\end{array}
$$

  where $full$ and $lastYear$ are boolean conditions over the initial state, whose exact form is irrelevant.

- **Frame problem:** we forgot to state in each branch of the conditional that the other variables do not change.

$$
\begin{array}{|l}
\hline
Enrol \\
\Delta CourseRequests \\
\hline
\neg\, full \Rightarrow r' = r + 1 \land c' = c - 1 \land w' = w \land n' = n \\
(full \land lastYear) \Rightarrow w' = w + 1 \land r' = r \land c' = c \land n' = n \\
(full \land \neg\, lastYear) \Rightarrow n' = n + 1 \land r' = r \land c' = c \land w' = w \\
\hline
\end{array}
$$

- Much longer, less perspicuous, and more difficult to change: Suppose a counter cw for the waiting list is added to the system; then all three conditionals need to be changed: one of them to indicate that cw is modified when the student is put on the waiting list, the other two just to say that $cw' = cw$.

Software Specification (Continued)

- **Strategy:**
  - Formalize these procedure specifications in the situation calculus.
  - Then reformulate the resulting solution to the frame problem to yield a mechanism for automatically generating this correct specification from the first one.

A. Borgida, J. Mylopoulos, and R. Reiter. "...and nothing else changes": The frame problem in procedure specifications. Technical report, Department of Computer Science, University of Toronto, 1992.

Complex Events

Joint work with with Hector Levesque and Fangzhen Lin.

Complex Events

- Want a way of expressing, and reasoning with, events defined in terms of *primitive events*.

$$\textbf{if } car\_in\_driveway \textbf{ then } drive \textbf{ else } walk.$$

$clear\_table \triangleq \textbf{while } [(\exists\, block) ontable(block)]\ remove\_a\_block.$

$remove\_a\_block \triangleq$
$(\pi\ block)[pickup(block); put\_on\_floor(block)].$

- Idea is to solve the frame problem for the primitive events. Complex events, defined in terms of the primitives, will inherit this solution.
- Solving the frame problem for primitive events: same trick as for database updates.
- Operators for defining complex events:
  - Sequence (;).
  - While loops.
  - Conditionals (**if then else**).
  - Nondeterministic choice of two events: Do α or do β.

$$\alpha \mid \beta.$$

  - Nondeterministic choice of arguments to events: The act of putting a block (any block) on A.

$$(\pi\ x) put(x, A)$$

.

Definitions of Complex Event Constructors

$Do(e, s, s')$ – Doing event e in state s leads to a state s'.

Abbreviations:

For $primitive$ events e:

$$Do(e, s, s') = Poss(e, s) \wedge s' = do(e, s).$$

$$Do(p?, s, s') = p \wedge s = s'.$$

$$Do([e_1; e_2], s, s') = (\exists\, s'').Do(e_1, s, s'') \wedge Do(e_2, s'', s').$$

$$Do([e_1 \mid e_2], s, s') = Do(e_1, s, s') \vee Do(e_2, s, s').$$

$$Do((\pi\ x)e, s, s') = (\exists\, x)Do(e, s, s').$$

$$\textbf{if } p \textbf{ then } e_1 \textbf{ else } e_2 = [p?; e_1] \mid [\neg p?; e_2].$$

$$\textbf{while } p\ e = [p?; e]^*; \neg p?.$$

$$Do(e^*, s, s') =$$
$$(\forall\, P).[(\forall\, s_1)P(s_1, s_1)] \wedge [(\forall\, s_1, s_2, s_3).P(s_1, s_2) \wedge Do(e, s_2, s_3) \supset P(s_1, s_3)]$$
$$\supset P(s, s').$$

Example: Controlling an Elevator

- Primitive events:

  $push(n),\ up(n),\ down(n),\ turnoff(n),\ open,\ close,\ no\_op.$

- Primitive fluents: $floor(s) = n,\ on(n, s).$

- Defined fluents: $next\_floor(n, s).$

- Primitive event preconditions:

$$Poss(up(n), s) \equiv floor(s) < n.$$

$$Poss(down(n), s) \equiv floor(s) > n.$$

$$Poss(open, s) \equiv true.$$

$$Poss(close, s) \equiv true.$$

$$Poss(push(n), s) \equiv \neg on(n, s).$$

$$Poss(turnoff(n), s) \equiv on(n, s).$$

$$Poss(no\_op, s) \equiv true.$$

- Primitive fluent values (same pattern as for database updates):

$$floor(do(e, s)) = m \equiv \{e = up(m) \vee e = down(m) \vee$$
$$floor(s) = m \wedge \neg(\exists\, n)e = up(n) \wedge \neg(\exists\, n)e = down(n)\}.$$

$$on(m, do(e, s)) \equiv e = push(m) \vee on(m, s) \wedge e \neq turnoff(m).$$

- Defined fluents.

$$next\_floor(n, s) \equiv on(n, s) \wedge$$
$$(\forall\, m).on(m, s) \supset |m - floor(s)| \geq |n - floor(s)|.$$

Elevator Example (Continued)

- Defining the complex events:

$$serve(n) \triangleq go\_floor(n); turnoff(n); open; close.$$

$$go\_floor(n) \triangleq \textbf{if } floor < n \textbf{ then } up(n) \textbf{ else } down(n).$$

$$serve\_a\_floor \triangleq (\pi\ n)[next\_floor(n); serve(n)].$$

$$control \triangleq [\textbf{while } (\exists\, n) on(n)\ serve\_a\_floor]; park.$$

$$park \triangleq \textbf{if } floor = 0 \textbf{ then } no\_op \textbf{ else } down(0); open.$$

- Initial state:

$$floor(S_0) = 3, \quad on(5, S_0), \quad on(1, S_0).$$

- Querying the theory:

$$Axioms \models (\exists\, s) Do(control, S_0, s).$$

- Successful proof might return

$$s = do(open, do(down(0), do(close, do(open, do(turnoff(5),\\ do(up(5), do(close, do(open, do(turnoff(1), do(down(1), S_0)))))))))$$

A Programming Language with a Difference

- A novel programming language, suitable for controlling discrete event systems like elevators and robots performing high level actions.
- First order (more or less) situation calculus semantics.
- Generalizes conventional imperative languages:
  - "Normal" imperative languages provide for procedure definitions which "bottom out" in simple operations on $machine$ states, e.g. $assignment$ statements.
  - Our complex events decompose into primitive events which may – and in interesting cases will – refer to events in some $external$ world (push an elevator button, pick up a block). "Machine states" = world states.
  - The "execution" of a complex event involves arbitrary first order reasoning; there is a collection of background axioms available, in addition to the complex event definitions.
    To execute

    $$\textbf{while } [(\exists\, block) ontable(block)]\ remove\_a\_block.$$

    requires inferring whether, in the current state,

    $$(\exists\, block) ontable(block).$$

Extensions in Progress

- Incorporating the agent's epistemic state (with Richard Scherl).
  - Knowing Mary's telephone number as a prerequisite for calling her.
  - Perception and its effects on the agent's epistemic state.
- Perception and its interaction with inference:
  Is block B on the table?
  1. $Axioms \models ontable(B, S)$?
  2. Look!
- Indexicals.
  "I am here now."
- Implementation.
  - Currently, there is a Prolog prototype.
  - Next: test on a radio-controlled robot-like device.

But Why the Situation Calculus?

- ... and not temporal logic, dynamic logic, etc.
- Because in the situation calculus, events are first class objects.
- Because the situation calculus is *first order*.
  - Simple, well-understood semantics, proof theory.
  - Availability of first-order theorem-provers.
- Because of a rich AI literature on the frame problem, nonmonotonic logics, planning.
- Because it is expressive enough to say everything these logics can say, and more (Pinto and Reiter).